

46-927, Spring 2021: Homework #1

Due 1:30PM Eastern, Wednesday, February 3, 2021.

Homework should be submitted on Gradescope before the deadline. Please submit your homework as a single PDF file. We recommend using a Jupyter notebook to prepare your homework. Use headings to clearly mark each problem in your PDF, and ensure it's legible once submitted to Gradescope.

Please post any questions on Piazza.

1. *Your very own AdaBoost.* In this problem, you will implement the essentials of the AdaBoost algorithm, and apply it to a classic example. You are using AdaBoost on a very simple classifier: a tree that is allowed to make exactly one split. Nevertheless, you will see that it achieves good performance once you boost it.

All the starter code is in `adaboost_starter_code.py`. To make your task easier, I've included several helper functions. These will handle the individual trees for you, as well as managing plotting of the results. Familiarize yourself with what these functions do.

You will also find partially completed templates for three functions: `my_adaboost`, `predict_ada`, `calculate_partial_dependence_x1`. Your job will be to fill in the missing pieces of these functions to make adaboost work.

Note: Throughout this problem you will be using the $y \in \{-1, 1\}$ convention.

- (a) Fill in the missing pieces of the `my_adaboost` function and the `predict_ada` function. The missing pieces are marked with `# TODO`. You'll be filling in the inner loop of the AdaBoost function; refer to the lecture slides. This function will return a list of trees and a vector of α_b weights. You will also fill in the prediction piece of the predict function, which computes the score $\sum_{b=1}^B \alpha_b \hat{f}^b(x)$ and thresholds it. The prediction function also computes running misclassification error across the number of trees, so that you can make a pretty plot later.
- (b) You will be applying your function to a classic boosting example. You have a cloud of points where the class depends on the distance from the origin (produced by the `get_circle_data` function). Even though your classifier can only make single splits along one axis, you will be able to combine them with boosting to do well on classifying whether points are inside a circle.

Generate a `train` and `test` set as in the example code with $n = 500$. Plot the first column on the x -axis and the second column on the y -axis), and color the points by their class (y).

Run your boosting method with $B = 1, 2, 3$, and use the `draw_boosted_trees` function to plot the decision boundaries and resulting scores. You can start to see how the circular classification problem is being approximated.

- (c) Run your boosting method with many more trees ($B = 250$). Plot the error on the training and test sets as a function of the number of trees (on one figure). The error as a function of number of trees is returned as part of the `predict_ada` function. If your AdaBoost implementation is working, your test and training errors should both be much improved (but different).

Plot the resulting boosted trees again using `draw_boosted_trees`. You'll see how the decision boundaries work together to produce a score that approximates the true circle.

What is your misclassification error on the test data for predicting whether a point is inside the circle?

- (d) Plot the partial dependence plot for `x1`. There is (slow) template code provided in `calculate_partial_dependence_x1`; you just need to fill in one piece. This will calculate the points that you should plot. Make your plot a line connecting these points. Expect the function to take a little while to execute.

2. *Boosting with XGboost*. In this problem, we will use a different data set and consider regression rather than classification. The homework folder contains the data file (`housing_data.csv`).

You will be using the `xgboost` library. To get started, install the `xgboost` package. This can be installed from `pip`, but you'll find some additional installation remarks at <https://pypi.python.org/pypi/xgboost/> depending on your system.

Load this data. Our goal is to predict the median house value from the other variables.

Split the data into three pieces, 60% to train, and two test sets of 20% each (call these your "test" set and your "final test" set). Use the following code:

```
from sklearn.model_selection import train_test_split
X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y, test_size=0.4, random_state=1)
X_test, X_final, y_test, y_final = train_test_split(
    X_tmp, y_tmp, test_size=0.5, random_state=10)
#You can discard X_tmp, y_tmp after this
```

Don't touch the “final test” data set in any way until the last part of this problem.

Throughout this problem, we will be evaluating the *mean absolute error* (MAE) of your predictions. This is easy to compute using `mean_absolute_error` from `sklearn.metrics`.

In an ideal world, we would actually optimize the MAE directly. However, XGBoost actually does something a little different from what we discussed in class, which requires a non-zero second derivative for the loss function, which makes optimizing the MAE directly difficult. We can talk about this more in class. For now, we'll optimize the MSE during tree construction (the default) and just consider tuning the MAE by cross-validation.

For the version you submit, please turn off all the output that the cv and training functions spit out by setting the `verbosity` parameter to 0.

- (a) Train boosting with `XGBRegressor` from the `xgboost` library. Use the default parameters:

```
{
    'learning_rate': 0.3,
    'max_depth': 6,
}
```

(The defaults may vary depending on which function in `xgboost` you use, so ensure you're using these.) What is the MAE of your predictions on the test set? (This is essentially the demo code I posted, but get it working with MAE.)

- (b) Train a random forest regression with 500 trees. How does the MAE compare to your default boosting approach?
- (c) Let's try tuning the number of trees for boosting. Use the faster built in `xgb.cv` function to cross-validate for up to 1000 trees. (Note: Set the `'eval_metric': 'mae'` in your parameter dictionary, so that you directly cross-validate for good MAE.)

Hint: Note: the `cv` and `train` functions require you to convert your data to DMatrix format, see the example from class.

Plot the resulting curve (the returned object has a `plot` function built in). What happens to the training and test error?

Pick the number of trees that gives the lowest cross-validated test error. Refit a model with this number of trees, and otherwise the default parameters. What is the MAE on the test set? How does this compare to the previous models?

- (d) Now, let's consider tuning the other parameters of the model, holding the number of trees fixed at the value you found. There is no great shortcut for this cross-validation, we will just use the cross-validation wrappers built into sklearn. To use these methods, you define a dictionary of parameters and lists of values. The function `GridSearchCV` will then cross-validate all combinations of these parameters. Warning: for many parameter values, this will be slow.

For this part of the problem, let's just try tuning the tree depth (`max_depth`) and the minimum child weight (`min_child_weight`, essentially a minimum number of points allowed in a leaf). Cross-validate for max depth in $\{3, 6, 9, 12\}$ and minimum weight in $\{1, 4, 8, 16\}$. Use `scoring='neg_mean_absolute_error'` so that you are directly cross-validating the error metric that you care about.

Which parameters do the best by this metric? What is the MAE performance on your test set for this winning model?

(Note: For the models in this and the next part, it would really make sense to retune the number of trees for your new parameters. But this assignment is getting long, so I'll skip that for now.)

- (e) One approach to cross-validating over a bigger parameter space is to do it by random sampling. The function `RandomizedSearchCV` will randomly sample sets of parameters in your parameter space and try them. This won't explore all combinations, but will save you both computation time and over fitting. Use `n_iter=16` so that you are searching the same number of combinations as last time. Again, make sure you are cross-validating the correct scoring measure, and use the number of trees that you found previously.

Try this using the following larger search space:

```
{
    'max_depth': [3,6,9,12],
    'min_child_weight': [1,4,8,16],
    'eta': [0.05, 0.1, 0.3, 0.5, 0.7],
    'gamma': [0, 10, 100, 1000],
    'subsample': [0.1, 0.3, 0.5, 1],
    'colsample_bylevel': [0.5, 0.7, 1]
}
```

Which parameters do the best in this random search? What is the MAE performance on your test set for this model?

- (f) Finally, recompute test performance for all your models on the “final test” set, which you have not touched until now. How do the new test errors compare to your previous test errors?

To think about: Does it really make sense to measure accuracy of housing values in absolute error? What would make more sense, and how could you optimize that instead? More to be said in class.